

C950 Task-2 WGUPS Write-Up

C950 Task-2 WGUPS Write-Up

(Task-2: The implementation phase of the WGUPS Routing Program).

(Zip your source code and upload it with this file)

Cole Linke

ID: 011917160

WGU Email: clink24@wgu.edu

7/10/25

C950 Data Structures and Algorithms II

C950 Task-2 WGUPS Write-Up

A. Hash Table

All Package Details at [08:00:00]								
ID	Truck	Street	City	Zipcode	Weight	Deadline	Delivered At	Status
1	1	195 W Oakland Ave	Salt Lake City	84115	21kg	10:30 AM	TBD	At Hub
2	3	2530 S 500 E	Salt Lake City	84106	44kg	EOD	TBD	At Hub
3	2	233 Canyon Rd	Salt Lake City	84103	2kg	EOD	TBD	At Hub
4	1	380 W 2880 S	Salt Lake City	84115	4kg	EOD	TBD	At Hub
5	1	410 S State St	Salt Lake City	84111	5kg	EOD	TBD	At Hub
6	2	3060 Lester St	West Valley City	84119	88kg	10:30 AM	TBD	Delayed until 9:05 AM
7	3	1330 2100 S	Salt Lake City	84106	8kg	EOD	TBD	At Hub
8	2	300 State St	Salt Lake City	84103	9kg	EOD	TBD	At Hub
9	3	300 State St	Salt Lake City	84103	2kg	EOD	TBD	At Hub
10	3	600 E 900 South	Salt Lake City	84105	1kg	EOD	TBD	At Hub
11	3	2600 Taylorsville Blvd	Salt Lake City	84118	1kg	EOD	TBD	At Hub
12	3	3575 W Valley Central Station Bus Loop	West Valley City	84119	1kg	EOD	TBD	At Hub
13	1	2010 W 500 S	Salt Lake City	84104	2kg	10:30 AM	TBD	At Hub
14	1	4300 S 1300 E	Millcreek	84117	88kg	10:30 AM	TBD	At Hub
15	1	4580 S 2300 E	Holladay	84117	4kg	9:00 AM	TBD	At Hub
16	1	4580 S 2300 E	Holladay	84117	88kg	10:30 AM	TBD	At Hub
17	3	3148 S 1100 W	Salt Lake City	84119	2kg	EOD	TBD	At Hub
18	2	1488 4800 S	Salt Lake City	84123	6kg	EOD	TBD	At Hub
19	1	177 W Price Ave	Salt Lake City	84115	37kg	EOD	TBD	At Hub
20	1	3595 Main St	Salt Lake City	84115	37kg	10:30 AM	TBD	At Hub
21	1	3595 Main St	Salt Lake City	84115	3kg	EOD	TBD	At Hub
22	3	6351 South 900 East	Murray	84121	2kg	EOD	TBD	At Hub
23	3	5100 South 2700 West	Salt Lake City	84118	5kg	EOD	TBD	At Hub
24	3	5025 State St	Murray	84107	7kg	EOD	TBD	At Hub
25	2	5383 South 900 East #104	Salt Lake City	84117	7kg	10:30 AM	TBD	Delayed until 9:05 AM
26	2	5383 South 900 East #104	Salt Lake City	84117	25kg	EOD	TBD	At Hub
27	3	1060 Dalton Ave S	Salt Lake City	84104	5kg	EOD	TBD	At Hub
28	2	2835 Main St	Salt Lake City	84115	7kg	EOD	TBD	Delayed until 9:05 AM
29	1	1330 2100 S	Salt Lake City	84106	2kg	10:30 AM	TBD	At Hub
30	1	300 State St	Salt Lake City	84103	1kg	10:30 AM	TBD	At Hub
31	1	3365 S 900 W	Salt Lake City	84119	1kg	10:30 AM	TBD	At Hub
32	2	3365 S 900 W	Salt Lake City	84119	1kg	EOD	TBD	Delayed until 9:05 AM
33	3	2530 S 500 E	Salt Lake City	84106	1kg	EOD	TBD	At Hub
34	1	4580 S 2300 E	Holladay	84117	2kg	10:30 AM	TBD	At Hub
35	3	1060 Dalton Ave S	Salt Lake City	84104	88kg	EOD	TBD	At Hub
36	2	2300 Parkway Blvd	West Valley City	84119	88kg	EOD	TBD	At Hub
37	1	410 S State St	Salt Lake City	84111	2kg	10:30 AM	TBD	At Hub
38	2	410 S State St	Salt Lake City	84111	9kg	EOD	TBD	At Hub
39	3	2010 W 500 S	Salt Lake City	84104	9kg	EOD	TBD	At Hub
40	1	380 W 2880 S	Salt Lake City	84115	45kg	10:30 AM	TBD	At Hub

B. Look-Up Functions

Look-up function code:

```
# TIME COMPLEXITY:  $O(N)$ , n = number of items in the bucket
# searches for an item with matching key in the hash table
# returns item if found, return None if not found
def search(self, key): 3 usages (3 dynamic)
    # get the bucket list where this key will be
    bucket = hash(key) % len(self.table)
    bucket_list = self.table[bucket]

    # search for the key in the bucket list
    for kv in bucket_list:
        if kv[0] == key:
            # if key exists, return its value
            return kv[1]
    # return None if key not found
    return None
```

C950 Task-2 WGUPS Write-Up

Look-up function in use:

```
# [2] General Details - Specific Package
elif user_input == '2':
    print('\nWhich package would you like to find:')
    try:
        # prompt user for package ID
        target_package = int(input('Package ID: '))
        # search hash table for the corresponding ID
        package = package_hash_table.search(target_package)

        # correct package 9's address at 10:20 AM
        time_change = datetime.timedelta(hours=10, minutes=20)
        package.address_update(time_change)

        # if package was found
        if package:
            # header section for target package
            print(f'\nPackage [{target_package}] details')
            print('-' * 120)
            print(f'{<ID>:5} | {<Truck>:6} | {<Street>:40} | {<City>:17} | {<Zipcode>:8} | {<Weight>:7}')
            print('-' * 120)

            # target package information
            print(f'{package.ID:5}'
                  f'| {package.truck_ID:6}'
                  f'| {package.address:40}'
                  f'| {package.city:17}'
                  f'| {package.zipcode:8}'
                  f'| {package.weight:7}'
                  f'| {package.deadline:9}'
                  f'| {str(package.delivery_time)[0:8] if package.delivery_time else 'N/A':13}')
            print('-' * 120)
        else:
            print(f'No package found with ID {target_package}') # package ID was not found
    except:
        print('Invalid Entry. Try again.')
```

C950 Task-2 WGUPS Write-Up

```
# [4] Time Specific - Single Package
elif user_input == '4':
    print('\nWhich package would you like to find:')

    try:
        # prompt user for package ID and search the hash table
        target_package = int(input('Package ID: '))
        package = package_hash_table.search(target_package)

        if package:
            try:
                # prompt user for time and parse input
                target_time = input('\nEnter a time [HH:MM:SS]: ')
                h, m, s, = map(int, target_time.strip().split(':'))
                time_change = datetime.timedelta(hours=h, minutes=m, seconds=s)

                # correct package 9's address at 10:20 AM
                package.address_update(time_change)

                # update status of target package
                package.delivery_update(time_change)

                # delayed packages
                package.delay_check(time_change)

                print(f'\nPackage {target_package}'s details at {target_time}')
                print('-' * 131)
                print(f"{'ID':<5}| {'Truck':<6}| {'Street':<40}| {'City':<17}| {'Zipcode':<8}| {'Weight':<7}|")
                print('-' * 131)

                print(f"{'package.ID':<5}"
                      f"| {'package.truck_ID':<6}"
                      f"| {'package.address':<40}"
                      f"| {'package.city':<17}"
                      f"| {'package.zipcode':<8}"
                      f"| {'package.weight':<7}"
                      f"| {'package.deadline':<9}"
                      f"| {'str(package.delivery_time)[0:8]"
                      if package.delivery_time and package.delivery_time <= time_change
                      else 'TBD'
                      f":<13}"
                      f"| {'package.status':<12}")
                print('-' * 131)
            except:
                print("Invalid time format. Use HH:MM:SS")
        else:
            print(f'No package found with ID {target_package}')
    except:
        print('Invalid Entry. Try again.')
```

C950 Task-2 WGUPS Write-Up

Look-up function results:

```
Package 10's details at 10:00:00
```

ID	Truck	Street	City	Zipcode	Weight	Deadline	Delivered At	Status
10	3	600 E 900 South	Salt Lake City	84105	1kg	EOD	TBD	At Hub

C. Original Code

ChainingHashTable.py

C950 Task-2 WGUPS Write-Up

```
1  # Source Citation:
2  # C950 Webinar 2 - Getting Greedy, who moved my data
3
4  # class to create hash table using chaining
5  class ChainingHashTable:
6
7      # TIME COMPLEXITY: O(N), n = initial capacity
8      # constructor to initialize the hash table with a capacity of 40
9      def __init__(self, initial_capacity=40):
10         # initialize table with an empty bucket list
11         self.table = []
12         for i in range(initial_capacity):
13             self.table.append([])
14
15
16     # TIME COMPLEXITY: O(N), n = number of items in the bucket
17     # insert new item into hash table
18     def insert(self, key, item):
19         # get the bucket list where this item will go
20         bucket = hash(key) % len(self.table)
21         bucket_list = self.table[bucket]
22
23         # update item if key already exists
24         for kv in bucket_list:
25             if kv[0] == key:
26                 kv[1] = item # update value
27                 return True
28
29         # if it does not exist, append new key-value pair
30         key_value = [key, item]
31         bucket_list.append(key_value)
32         return True
33
34
35     # TIME COMPLEXITY: O(N), n = number of items in the bucket
36     # searches for an item with matching key in the hash table
37     # returns item if found, return None if not found
38     def search(self, key):
39         # get the bucket list where this key will be
40         bucket = hash(key) % len(self.table)
41         bucket_list = self.table[bucket]
42
43         # search for the key in the bucket list
44         for kv in bucket_list:
45             if kv[0] == key:
46                 # if key exists, return its value
47                 return kv[1]
48         # return None if key not found
49         return None
50
51
52     # TIME COMPLEXITY: O(N), n = number of items in the bucket
53     # removes an item with matching key from the hash table
54     def remove(self, key):
55         # get the bucket list where this key will be removed
56         bucket = hash(key) % len(self.table)
57         bucket_list = self.table[bucket]
58
59         # remove the item from the bucket list if it is present
60         for kv in bucket_list:
61             if kv[0] == key:
62                 bucket_list.remove([kv[0], kv[1]])
63                 # return true if kv is found and removed
64                 return True
65         # return false if key is not found
66         return False
67
68
69     # TIME COMPLEXITY: O(N), n = total number of key-value pairs
70     # print all values stored in hash table (testing purposes)
71     def print_table(self):
72         for i in range(len(self.table)):
73             bucket = self.table[i]
74             if bucket:
75                 print(' '.join(str(kv[1]) for kv in bucket))
76
77
78     # TIME COMPLEXITY: O(n), n = total number of key-value pairs
79     # retrieve all items stored in the hash table
80     def get_all_items(self):
81         all_items = []
82         for bucket in self.table:
83             for key, item in bucket:
84                 all_items.append(item)
85         return all_items
86
87
88     # TIME COMPLEXITY: O(1)
89     def __str__(self):
90         return f'chaining hash table: {self.table}'
91     # TIME COMPLEXITY: O(1)
92     def __repr__(self):
93         return f'chaining hash table: {self.table}'
94
```

C950 Task-2 WGUPS Write-Up

CSV_Import.py

```
# Source Citation:
# CSV - CSV file reading and writing. (n.d.). Python Documentation. https://docs.python.org/3/library/csv.html#csv.reader

import csv
from ChainingHashTable import ChainingHashTable
from Package import Package

# TIME COMPLEXITY: O(n), n = number of lines in CSV file
# loads the distance and address data from distance.csv and address.csv
def load_csv_files():
    with open('CSV/Distance_Table.csv') as csvfile:
        # reads Distance_Table.csv and puts it into a list of rows
        distance_reader = list(csv.reader(csvfile))

    with open('CSV/Address_Table.csv') as csvfile1:
        # reads Address_Table.csv and puts it into a list of rows
        address_reader = list(csv.reader(csvfile1))

    # return both csv tables
    return distance_reader, address_reader

# TIME COMPLEXITY: O(N), n = number of packages in CSV file
# loads package data from csv and stores it in a hash table
def load_packages(filename):
    # initialize chaining hash table
    package_hash_table = ChainingHashTable()

    # Open the CSV file
    with open(filename, encoding='utf-8-sig') as packages:
        # Create an object to read CSV file
        package_reader = csv.reader(packages, delimiter=',')

        # process each row in csv file
        for package in package_reader:
            if len(package) < 7:
                continue # skips rows with missing required fields

            # Extract package fields from the row
            pack_ID = int(package[0])
            address = package[1]
            city = package[2]
            state = package[3]
            zipcode = package[4]
            deadline = package[5]
            # change kilos to kg for formatting reasons
            weight = package[6].replace(' Kilos', 'kg')
            notes = package[7] if len(package) > 7 else ''

            # create package object and insert into hash table
            package_hash_table.insert(pack_ID, Package(pack_ID, address, city, state, zipcode, deadline, weight, notes))

    return package_hash_table
```

Package.py

C950 Task-2 WGUPS Write-Up

```
1  # class to create a package object
2  import datetime
3
4
5  class Package:
6
7      # TIME COMPLEXITY: O(1)
8      # constructor to initialize a package object with all relevant information
9      def __init__(self, ID, address, city, state, zipcode, deadline, weight, notes, status='At Hub', depart_time=None, delivery_time=None, truck_ID=None):
10         self.ID = ID
11         self.address = address
12         self.city = city
13         self.state = state
14         self.zipcode = zipcode
15         self.deadline = deadline
16         self.weight = weight
17         self.notes = notes
18         self.status = status
19         self.depart_time = depart_time
20         self.delivery_time = delivery_time
21         self.truck_ID = truck_ID
22
23
24      # TIME COMPLEXITY: O(1)
25      # returns package information as a string
26      def __repr__(self):
27         return (
28             f"ID: {self.ID} \t "
29             f"Address: {self.address} \t "
30             f"City: {self.city} \t "
31             f"State: {self.state} \t "
32             f"Zip: {self.zipcode} \t "
33             f"Deadline: {self.deadline} \t "
34             f"Weight: {self.weight} \t "
35             f"Notes: {self.notes} \t "
36             f"Status: {self.status} \t "
37             f"Departure Time: {self.depart_time} \t "
38             f"Delivery Time: {self.delivery_time} \n" %
39             (self.ID, self.address, self.city, self.state, self.zipcode, self.deadline, self.weight, self.notes, self.status,
40              self.depart_time, self.delivery_time))
41
42
43      # TIME COMPLEXITY: O(1)
44      def __str__(self):
45         return (
46             f"ID: {self.ID} \t "
47             f"Address: {self.address} \t "
48             f"City: {self.city} \t "
49             f"State: {self.state} \t "
50             f"Zip: {self.zipcode} \t "
51             f"Deadline: {self.deadline} \t "
52             f"Weight: {self.weight} \t "
53             f"Notes: {self.notes} \t "
54             f"Status: {self.status} \t "
55             f"Departure Time: {self.depart_time} \t "
56             f"Delivery Time: {self.delivery_time} \n" %
57             (self.ID, self.address, self.city, self.state, self.zipcode, self.deadline, self.weight, self.notes,
58              self.status,
59              self.depart_time, self.delivery_time))
60
61
62      # TIME COMPLEXITY: O(1)
63      # updates the status of package based on a given time
64      def delivery_update(self, time_change):
65         # not yet left the hub
66         if time_change < self.depart_time:
67             self.status = 'At Hub'
68         # out for delivery
69         elif self.depart_time <= time_change < self.delivery_time:
70             self.status = 'En Route'
71         # has been delivered
72         elif time_change >= self.delivery_time:
73             self.status = 'Delivered'
74
75      # TIME COMPLEXITY: O(1)
76      # corrects the address of package 9 after 10:20 AM
77      def address_update(self, time_change):
78         # check that package ID is 9 and time is after 10:20 AM
79         if self.ID == 9 and time_change >= datetime.timedelta(hours=10, minutes=20):
80             self.address = '410 S State St'
81             self.city = 'Salt Lake City'
82             self.state = 'UT'
83             self.zipcode = '84111'
84
85      # TIME COMPLEXITY: O(1)
86      # sets status of delayed packaged to delays
87      def delay_check(self, time_change):
88         # check for certain package IDs and if they have not left yet
89         if self.ID in {6, 25, 28, 32}:
90             delay_time = datetime.timedelta(hours=9, minutes=5)
91             if time_change <= delay_time:
92                 self.status = 'Delayed until 9:05 AM'
93
```

Truck.py

```
1  # class to create a truck object
2  class Truck:
3
4      # TIME COMPLEXITY: O(1)
5      # initializes a truck object with required attributes
6      def __init__(self, ID, speed, package, mileage, address, depart_time):
7          self.ID = ID
8          self.speed = speed
9          self.package = package
10         self.mileage = mileage
11         self.address = address
12         self.depart_time = depart_time
13         self.return_time = None          # set with return_to_hub function
14
15
16     # TIME COMPLEXITY: O(1)
17     # returns truck's information as a string
18     def __str__(self):
19         return "%s, %s, %s, %s, %s" % (self.speed, self.package, self.mileage,
20                                         self.address, self.depart_time)
21
22
23     # TIME COMPLEXITY: O(1)
24     def __repr__(self):
25         return "%s, %s, %s, %s, %s" % (self.speed, self.package, self.mileage,
26                                         self.address, self.depart_time)
```

Ul.py #1

C950 Task-2 WGUPS Write-Up

```
1  # Sources Cited:
2  # Built-in Functions. (n.d.). Python Documentation. https://docs.python.org/3/library/functions.html#sorted
3  # ANSI Escape sequences. (n.d.). ASCII-Table. https://web.archive.org/web/20201214113226/http://ascii-table.com/ansi-escape-sequences.php
4  # Sorting techniques. (n.d.). Python Documentation. https://docs.python.org/3/howto/sorting.html
5  # datetime - Basic date and time types. (n.d.). Python Documentation. https://docs.python.org/3/library/datetime.html#strptime-and-strftime-behavior
6
7
8  import datetime
9
10
11 # TIME COMPLEXITY: O(n), n = number of packages
12 # this method runs a terminal-based interface
13 # allows the user to return package data for general or time-specific queries
14 def run_ui(package_hash_table):
15
16     # flag to keep UI looping
17     is_running = True
18
19     # TIME COMPLEXITY: O(n log (n)), Timsort sorting algorithm
20     while is_running:
21
22         # Print main menu
23         print('\n\033[1m--- EOD Package Details ---\033[0m')
24         '\n[1] Get all package details'
25         '\n[2] Get a specific package's details'
26         '\n\033[1m--- Time Specific Package Details ---\033[0m'
27         '\n[3] Get all package details'
28         '\n[4] Get a specific package's details'
29         '\n-----'
30         '\n[!] Quit Program')
31
32     try:
33
34         # prompt user for menu selection
35         user_input = input(' [ ]: ')
36
37         # [!] Quit Program
38         if user_input == '!':
39             print('Quitting program')
40             is_running = False # flips flag, ends loop
41
42         # [1] General Details - All Packages
43         elif user_input == '1':
44
45             # header section for package table
46             print('\nAll Package Details:')
47             print('-' * 120)
48             print(f'{ "ID":<5} | { "Truck":<6} | { "Street":<40} | { "City":<17} | { "Zipcode":<8} | { "Weight":<7} | { "Deadline":<9} | { "Delivered At":<13}"')
49             print('-' * 120)
50
51             # TIME COMPLEXITY: O(n log (n)), Timsort sorting algorithm
52             # iterates through each package in hash table, sorts by ID
53             for package in sorted(package_hash_table.get_all_items(), key=lambda p: p.ID):
54
55                 # correct package 9's address at 10:20 AM
56                 time_change = datetime.timedelta(hours=10, minutes=20)
57                 package.address_update(time_change)
58
59                 # print package information
60                 print(f'{package.ID:<5}" # format output spacing for readability
61                     f'{package.truck_ID:<6}"
62                     f'{package.address:<40}"
63                     f'{package.city:<17}"
64                     f'{package.zipcode:<8}"
65                     f'{package.weight:<7}"
66                     f'{package.deadline:<9}"
67                     # format time, output 'N/A' if the package does not have a deliver_time
68                     f'{str(package.delivery_time)[0:8] if package.delivery_time else "N/A":<13}"')
69                 print('-' * 120)
70
71         # [2] General Details - Specific Package
72         elif user_input == '2':
73             print('\nWhich package would you like to find:')
74             try:
75                 # prompt user for package ID
76                 target_package = int(input('Package ID: '))
77                 # search hash table for the corresponding ID
78                 package = package_hash_table.search(target_package)
79
80                 # correct package 9's address at 10:20 AM
81                 time_change = datetime.timedelta(hours=10, minutes=20)
82                 package.address_update(time_change)
83
84                 # if package was found
85                 if package:
86                     # header section for target package
87                     print(f'\nPackage [{target_package}] details')
88                     print('-' * 120)
89                     print(f'{ "ID":<5} | { "Truck":<6} | { "Street":<40} | { "City":<17} | { "Zipcode":<8} | { "Weight":<7} | { "Deadline":<9} | { "Delivered At":<13}"')
90                     print('-' * 120)
91
92                     # target package information
93                     print(f'{package.ID:<5}"
94                         f'{package.truck_ID:<6}"
95                         f'{package.address:<40}"
96                         f'{package.city:<17}"
97                         f'{package.zipcode:<8}"
98                         f'{package.weight:<7}"
99                         f'{package.deadline:<9}"
100                        f'{str(package.delivery_time)[0:8] if package.delivery_time else "N/A":<13}"')
101                     print('-' * 120)
102                 else:
103                     print(f'No package found with ID {target_package}') # package ID was not found
104             except:
105                 print('Invalid Entry. Try again.')
```

UI.py #2

C950 Task-2 WGUPS Write-Up

```
1063     print('Invalid Entry. Try again.')
1064
1065     # [3] Time Specific - All Packages
1066     elif user_input == '3':
1067         try:
1068             # prompt user for time
1069             target_time = input('\nEnter a time [HH:MM:SS]: ')
1070             # parse time input
1071             h, m, s = map(int, target_time.strip().split(':'))
1072             time_change = datetime.timedelta(hours=h, minutes=m, seconds=s)
1073
1074             # update all packages based on time given
1075             for package in package_hash_table.get_all_items():
1076                 # correct package 9's address at 10:20 AM
1077                 package.address_update(time_change)
1078                 package.delivery_update(time_change)
1079                 # delayed packages
1080                 package.delay_check(time_change)
1081
1082             print(f'\nAll Package Details at [{target_time}]')
1083             print('-' * 131)
1084             print(f'{"ID":<5}| {"Truck":<6}| {"Street":<40}| {"City":<17}| {"Zipcode":<8}| {"Weight":<7}| {"Deadline":<9}| {"Delivered At":<13}| {"Status":<12}|')
1085             print('-' * 131)
1086
1087             for package in sorted(package_hash_table.get_all_items(), key=lambda p: p.ID):
1088                 print(f'{"package.ID":<5}|')
1089                 f'{"package.truck_ID":<6}|'
1090                 f'{"package.address":<40}|'
1091                 f'{"package.city":<17}|'
1092                 f'{"package.zipcode":<8}|'
1093                 f'{"package.weight":<7}|'
1094                 f'{"package.deadline":<9}|'
1095                 # if a package has not been delivered yet, output 'TBD'
1096                 f'| {str(package.delivery_time)[0:8]}')
1097                 if package.delivery_time and package.delivery_time <= time_change:
1098                     else 'TBD'
1099                 f'| {str(package.delivery_time)[0:8]}')
1100                 # add status to the output list
1101                 f'| {package.status:<12}|')
1102             print('-' * 131)
1103         except:
1104             print('Invalid time format. Use HH:MM:SS')
1105
1106     # [4] Time Specific - Single Package
1107     elif user_input == '4':
1108         print('\nWhich package would you like to find:')
1109
1110         try:
1111             # prompt user for package ID and search the hash table
1112             target_package = int(input('Package ID: '))
1113             package = package_hash_table.search(target_package)
1114
1115             if package:
1116                 try:
1117                     # prompt user for time and parse input
1118                     target_time = input('\nEnter a time [HH:MM:SS]: ')
1119                     h, m, s = map(int, target_time.strip().split(':'))
1120                     time_change = datetime.timedelta(hours=h, minutes=m, seconds=s)
1121
1122                     # correct package 9's address at 10:20 AM
1123                     package.address_update(time_change)
1124
1125                     # update status of target package
1126                     package.delivery_update(time_change)
1127
1128                     # delayed packages
1129                     package.delay_check(time_change)
1130
1131                     print(f'\nPackage {target_package}'s details at {target_time}')
1132                     print('-' * 131)
1133                     print(f'{"ID":<5}| {"Truck":<6}| {"Street":<40}| {"City":<17}| {"Zipcode":<8}| {"Weight":<7}| {"Deadline":<9}| {"Delivered At":<13}| {"Status":<12}|')
1134                     print('-' * 131)
1135
1136                     print(f'{"package.ID":<5}|')
1137                     f'{"package.truck_ID":<6}|'
1138                     f'{"package.address":<40}|'
1139                     f'{"package.city":<17}|'
1140                     f'{"package.zipcode":<8}|'
1141                     f'{"package.weight":<7}|'
1142                     f'{"package.deadline":<9}|'
1143                     f'| {str(package.delivery_time)[0:8]}')
1144                     if package.delivery_time and package.delivery_time <= time_change:
1145                         else 'TBD'
1146                     f'| {str(package.delivery_time)[0:8]}')
1147                     f'| {package.status:<12}|')
1148                     print('-' * 131)
1149                 except:
1150                     print('Invalid time format. Use HH:MM:SS')
1151             else:
1152                 print(f'No package found with ID {target_package}')
1153         except:
1154             print('Invalid Entry. Try again.')
1155
1156     # catch all invalid menu entries
1157     else:
1158         print('Invalid entry, Trying again')
1159     except:
1160         print('Invalid entry, Trying again')
```

Main.py

C950 Task-2 WGUPS Write-Up

```
1  # Cole Linke
2  # Student ID: 011917160
3  # C950 WGUPS Routing Program Implementation
4
5  import datetime
6
7  # import modules
8  from CSV_Import import load_csv_files, load_packages
9  from Delivery import delivery, return_to_hub, distance_between, load_address
10 from Truck import Truck
11 from UI import run_ui
12
13
14 # TIME COMPLEXITY: O(1)
15 # create truck objects
16
17 # create and assign truck1 with packages, leaves at 8:00:01 AM
18 truck1 = Truck(1, 18, [1, 4, 5, 13, 14, 15, 16, 19, 20, 21, 29, 30, 31, 34, 37, 40],
19                0.0, '4001 South 700 East', datetime.timedelta(hours=8, minutes=0, seconds=1))
20
21 # create and assign truck2 with packages, leaves at 9:05:01 AM
22 truck2 = Truck(2, 18, [3, 6, 8, 18, 25, 26, 28, 32, 36, 38],
23                0.0, '4001 South 700 East', datetime.timedelta(hours=9, minutes=5, seconds=1))
24
25 # create and assign truck3 with packages, leaves at 10:20:01 AM
26 truck3 = Truck(3, 18, [2, 7, 9, 10, 11, 12, 17, 22, 23, 24, 27, 33, 35, 39],
27                0.0, '4001 South 700 East', datetime.timedelta(hours=10, minutes=20, seconds=1))
28
29
30 # TIME COMPLEXITY: O(n), n = number of lines in CSV file
31 # load CSV data
32
33 # load distance and address data
34 distance_reader, address_reader = load_csv_files()
35 # load package data into hash table
36 package_hash_table = load_packages('CSV/Package_Table.csv')
37
38
39 # TIME COMPLEXITY: O(n^2), n = number of packages on the truck
40 # deliver packages
41
42 # run delivery algo for both truck1 and truck2
43 delivery(truck1, package_hash_table, distance_reader, address_reader)
44 delivery(truck2, package_hash_table, distance_reader, address_reader)
45 # make truck3 wait until its either 10:20:01 AM or another truck has returned to the hub
46 truck3.depart_time = max(datetime.timedelta(hours=10, minutes=20, seconds=1), min(truck1.return_time, truck2.return_time))
47 # run delivery algo for truck3
48 delivery(truck3, package_hash_table, distance_reader, address_reader)
49
50
51 # print author header
52 print('\nCole Linke')
53 print('\nID: 011917160')
54
55 # print formatted summary for each truck
56 print('\n\033[1m--- Truck One Details ---\033[0m')
57 print(f'Depart Time: {truck1.depart_time}') # time truck left the hub
58 print(f'Return Time: {truck1.return_time}') # time truck returned to the hub
59 print(f'Total Mileage: {round(truck1.mileage, 1)}') # mileage for select truck
60 print('\n\033[1m--- Truck Two Details ---\033[0m')
61 print(f'Depart Time: {truck2.depart_time}')
62 print(f'Return Time: {truck2.return_time}')
63 print(f'Total Mileage: {round(truck2.mileage, 1)}')
64 print('\n\033[1m--- Truck Three Details ---\033[0m')
65 print(f'Depart Time: {truck3.depart_time}')
66 print(f'Return Time: {truck3.return_time}')
67 print(f'Total Mileage: {round(truck3.mileage, 1)}')
68 print('-' * 27)
69 total_mileage = round(truck1.mileage + truck2.mileage + truck3.mileage, 1)
70 print(f'Total Truck Mileage: {total_mileage}') # total mileage for all 3 trucks
71
72
73 # TIME COMPLEXITY: O(n)
74 # run user interface
75 run_ui(package_hash_table)
76
77
```

Delivery.py

C950 Task-2 WGUPS Write-Up

```

1 import datetime
2
3
4 # TIME COMPLEXITY: O(n), n = number of addresses in address_reader
5 # uses target_address to find address ID in address_reader
6 def load_address(target_address, address_reader):
7     for row in address_reader:
8         if target_address in row[2]:
9             # If the address is found, return the corresponding address ID
10             return int(row[0])
11         # address not found
12     return None
13
14
15 # TIME COMPLEXITY: O(1)
16 # finds the distance between two addresses
17 def distance_between(a, b, distance_reader):
18     # get distance from a to b in the distance matrix
19     distance = distance_reader[a][b]
20     # if distance is empty, get distance from b to a
21     if distance == '':
22         distance = distance_reader[b][a]
23     # return distance
24     return float(distance)
25
26
27 # TIME COMPLEXITY: O(n), n = number of addresses in address_reader
28 # calculates distance for the truck to return to the hub and updates truck mileage and return time
29 def return_to_hub(truck, current_time, distance_reader, address_reader):
30     hub_address = '4001 South 700 East'
31
32     # find address ID for truck's current address and the hub address
33     return_distance = distance_between(
34         load_address(truck.address, address_reader),
35         load_address(hub_address, address_reader),
36         distance_reader)
37
38     # add return distance to truck's total mileage
39     truck.mileage += return_distance
40     # update truck's current address to hub address
41     truck.address = hub_address
42     # calculate and update the truck's return time
43     truck.return_time = current_time + datetime.timedelta(hours=return_distance / truck.speed)
44     return truck.return_time
45
46
47 # TIME COMPLEXITY: O(n^2), n = number of packages on each truck
48 # uses nearest neighbor algorithm, delivers packages in order of shortest distance from current location
49 def delivery(truck, package_hash_table, distance_reader, address_reader):
50     # empty list to hold undelivered packages
51     undelivered_packages = []
52
53     # loop through each package ID assigned to this truck
54     for pack_ID in truck.packages:
55         # loop up the package in the hash table
56         package = package_hash_table.search(pack_ID)
57         if package is not None:
58             # add package to the undelivered list
59             undelivered_packages.append(package)
60         else:
61             # print warning if package ID was not found
62             print(f'Package {pack_ID} not found')
63
64     # clear the truck's package list to reorder during delivery
65     truck.package.clear()
66
67     # set current time to the truck's departure time
68     current_time = truck.depart_time
69
70     # continue while there are still undelivered packages
71     while len(undelivered_packages) > 0:
72         # initialize shortest distance to infinity
73         shortest_distance = float('inf')
74         # placeholder for the next closest package
75         next_package = None
76
77         # iterate through all undelivered packages to find the closest one
78         for package in undelivered_packages:
79             # find the distance between the truck's current address and the package's address
80             dist = distance_between(
81                 load_address(truck.address, address_reader),
82                 load_address(package.address, address_reader),
83                 distance_reader)
84
85             # save the shortest package and distance
86             if dist < shortest_distance:
87                 shortest_distance = dist
88                 next_package = package
89
90         # add the saved package ID to the truck's delivery order
91         truck.package.append(next_package.ID)
92
93         # add the distance to the truck's mileage
94         truck.mileage += shortest_distance
95
96         # calculate travel time
97         travel_time = datetime.timedelta(hours=shortest_distance / truck.speed)
98         # add travel time to current time
99         current_time += travel_time
100
101         # update truck's current location to the delivered package's location
102         truck.address = next_package.address
103
104         # save the departure and delivery time of the package
105         next_package.depart_time = truck.depart_time
106         next_package.delivery_time = current_time
107         next_package.truck_ID = truck.ID
108
109         # remove the delivered package from the undelivered list
110         undelivered_packages.remove(next_package)
111
112     # once all packages are delivered, return truck back to the hub
113     return_to_hub(truck, current_time, distance_reader, address_reader)

```

C1. Identification Information

```
1  # Cole Linke
2  # Student ID: 011917160
3  # C950 WGUPS Routing Program Implementation
4
5  import datetime
6
7  # import modules
8  from CSV_Import import load_csv_files, load_packages
9  from Delivery import delivery, return_to_hub, distance_between, load_address
10 from Truck import Truck
11 from UI import run_ui
12
13
14 # TIME COMPLEXITY: O(1)
15 # create truck objects
16
17 # create and assign truck1 with packages, leaves at 8:00AM
18 truck1 = Truck( ID: 1, speed: 18, package: [1, 4, 5, 13, 14, 15, 16, 19, 20, 21, 29, 30, 31, 34, 37, 40],
19                mileage: 0.0, address: '4001 South 700 East', datetime.timedelta(hours=8))
20
21 # create and assign truck2 with packages, leaves at 9:05AM
22 truck2 = Truck( ID: 2, speed: 18, package: [3, 6, 8, 18, 25, 26, 28, 32, 36, 38],
23                mileage: 0.0, address: '4001 South 700 East', datetime.timedelta(hours=9, minutes=5))
24
25 # create and assign truck3 with packages, leaves at 10:20AM
26 truck3 = Truck( ID: 3, speed: 18, package: [2, 7, 9, 10, 11, 12, 17, 22, 23, 24, 27, 33, 35, 39],
27                mileage: 0.0, address: '4001 South 700 East', datetime.timedelta(hours=10, minutes=20))
28
```

C2. Process and Flow Comments

Comment and Flow example:

C950 Task-2 WGUPS Write-Up

```
# TIME COMPLEXITY: O(N), n = number of items in the bucket
# insert new item into hash table
def insert(self, key, item): 1 usage
    # get the bucket list where this item will go
    bucket = hash(key) % len(self.table)
    bucket_list = self.table[bucket]

    # update item if key already exists
    for kv in bucket_list:
        if kv[0] == key:
            kv[1] = item # update value
            return True

    # if it does not exist, append new key-value pair
    key_value = [key, item]
    bucket_list.append(key_value)
    return True

# TIME COMPLEXITY: O(N), n = number of items in the bucket
# searches for an item with matching key in the hash table
# returns item if found, return None if not found
def search(self, key): 3 usages (3 dynamic)
    # get the bucket list where this key will be
    bucket = hash(key) % len(self.table)
    bucket_list = self.table[bucket]

    # search for the key in the bucket list
    for kv in bucket_list:
        if kv[0] == key:
            # if key exists, return its value
            return kv[1]

    # return None if key not found
    return None
```


Comment and Flow example 2:

```
# TIME COMPLEXITY: O(n), n = number of addresses in address_reader
# uses target_address to find address ID in address_reader
def load_address(target_address, address_reader): 5 usages
    for row in address_reader:
        if target_address in row[2]:
            # if the address is found, return the corresponding address ID
            return int(row[0])
    # address not found
    return None
```

D. Interface

Interface Ex 1:

```
Cole Linke
ID: 011917160

--- Truck One Details ---
Depart Time: 8:00:00
Return Time: 9:53:20
Total Mileage: 34.0
--- Truck Two Details ---
Depart Time: 9:05:00
Return Time: 11:09:40
Total Mileage: 37.4
--- Truck Three Details ---
Depart Time: 10:20:01
Return Time: 13:06:41
Total Mileage: 50.0
-----
Total Truck Mileage: 121.4

--- EOD Package Details ---
[1] Get all package details
[2] Get a specific package's details
--- Time Specific Package Details ---
[3] Get all package details
[4] Get a specific package's details
-----
[!] Quit Program
[ ]:
```

C950 Task-2 WGUPS Write-Up

Interface Ex 2:

```
--- EOD Package Details ---
[1] Get all package details
[2] Get a specific package's details
--- Time Specific Package Details ---
[3] Get all package details
[4] Get a specific package's details
-----
[!] Quit Program
[ ]: 4

Which package would you like to find:
Package ID: 10

Enter a time [HH:MM:SS]: 08:30:00

Package 10's details at 08:30:00
-----
ID   | Truck | Street                | City          | Zipcode | Weight | Deadline | Delivered At | Status
-----
10   | 3      | 600 E 900 South       | Salt Lake City | 84105   | 1kg    | EOD      | TBD          | At Hub
-----
```

Interface Ex 3:

```
--- EOD Package Details ---
[1] Get all package details
[2] Get a specific package's details
--- Time Specific Package Details ---
[3] Get all package details
[4] Get a specific package's details
-----
[!] Quit Program
[ ]: !

Quitting program

Process finished with exit code 0
```

C950 Task-2 WGUPS Write-Up

D1. First Status Check

All Package Details at [08:45:00]								
ID	Truck	Street	City	Zipcode	Weight	Deadline	Delivered At	Status
1	1	195 W Oakland Ave	Salt Lake City	84115	21kg	10:30 AM	8:40:41	Delivered
2	3	2530 S 500 E	Salt Lake City	84106	44kg	EOD	TBD	At Hub
3	2	233 Canyon Rd	Salt Lake City	84103	2kg	EOD	TBD	At Hub
4	1	380 W 2880 S	Salt Lake City	84115	4kg	EOD	8:37:01	Delivered
5	1	410 S State St	Salt Lake City	84111	5kg	EOD	TBD	En Route
6	2	3060 Lester St	West Valley City	84119	88kg	10:30 AM	TBD	Delayed until 9:05 AM
7	3	1330 2100 S	Salt Lake City	84106	8kg	EOD	TBD	At Hub
8	2	300 State St	Salt Lake City	84103	9kg	EOD	TBD	At Hub
9	3	300 State St	Salt Lake City	84103	2kg	EOD	TBD	At Hub
10	3	600 E 900 South	Salt Lake City	84105	1kg	EOD	TBD	At Hub
11	3	2600 Taylorsville Blvd	Salt Lake City	84118	1kg	EOD	TBD	At Hub
12	3	3575 W Valley Central Station bus Loop	West Valley City	84119	1kg	EOD	TBD	At Hub
13	1	2010 W 500 S	Salt Lake City	84104	2kg	10:30 AM	TBD	En Route
14	1	4300 S 1300 E	Millcreek	84117	88kg	10:30 AM	8:06:21	Delivered
15	1	4580 S 2300 E	Holladay	84117	4kg	9:00 AM	8:13:01	Delivered
16	1	4580 S 2300 E	Holladay	84117	88kg	10:30 AM	8:13:01	Delivered
17	3	3148 S 1100 W	Salt Lake City	84119	2kg	EOD	TBD	At Hub
18	2	1488 4800 S	Salt Lake City	84123	6kg	EOD	TBD	At Hub
19	1	177 W Price Ave	Salt Lake City	84115	37kg	EOD	8:31:21	Delivered
20	1	3595 Main St	Salt Lake City	84115	37kg	10:30 AM	8:29:41	Delivered
21	1	3595 Main St	Salt Lake City	84115	3kg	EOD	8:29:41	Delivered
22	3	6351 South 900 East	Murray	84121	2kg	EOD	TBD	At Hub
23	3	5100 South 2700 West	Salt Lake City	84118	5kg	EOD	TBD	At Hub
24	3	5025 State St	Murray	84107	7kg	EOD	TBD	At Hub
25	2	5383 South 900 East #104	Salt Lake City	84117	7kg	10:30 AM	TBD	Delayed until 9:05 AM
26	2	5383 South 900 East #104	Salt Lake City	84117	25kg	EOD	TBD	At Hub
27	3	1060 Dalton Ave S	Salt Lake City	84104	5kg	EOD	TBD	At Hub
28	2	2835 Main St	Salt Lake City	84115	7kg	EOD	TBD	Delayed until 9:05 AM
29	1	1330 2100 S	Salt Lake City	84106	2kg	10:30 AM	TBD	En Route
30	1	300 State St	Salt Lake City	84103	1kg	10:30 AM	TBD	En Route
31	1	3365 S 900 W	Salt Lake City	84119	1kg	10:30 AM	TBD	En Route
32	2	3365 S 900 W	Salt Lake City	84119	1kg	EOD	TBD	Delayed until 9:05 AM
33	3	2530 S 500 E	Salt Lake City	84106	1kg	EOD	TBD	At Hub
34	1	4580 S 2300 E	Holladay	84117	2kg	10:30 AM	8:13:01	Delivered
35	3	1060 Dalton Ave S	Salt Lake City	84104	88kg	EOD	TBD	At Hub
36	2	2300 Parkway Blvd	West Valley City	84119	88kg	EOD	TBD	At Hub
37	1	410 S State St	Salt Lake City	84111	2kg	10:30 AM	TBD	En Route
38	2	410 S State St	Salt Lake City	84111	9kg	EOD	TBD	At Hub
39	3	2010 W 500 S	Salt Lake City	84104	9kg	EOD	TBD	At Hub
40	1	380 W 2880 S	Salt Lake City	84115	45kg	10:30 AM	8:37:01	Delivered

C950 Task-2 WGUPS Write-Up

D2. Second Status Check

All Package Details at [09:45:00]

ID	Truck	Street	City	Zipcode	Weight	Deadline	Delivered At	Status
1	1	195 W Oakland Ave	Salt Lake City	84115	21kg	10:30 AM	8:40:41	Delivered
2	3	2530 S 500 E	Salt Lake City	84106	44kg	EOD	TBD	At Hub
3	2	233 Canyon Rd	Salt Lake City	84103	2kg	EOD	TBD	En Route
4	1	380 W 2880 S	Salt Lake City	84115	4kg	EOD	8:37:01	Delivered
5	1	410 S State St	Salt Lake City	84111	5kg	EOD	9:04:21	Delivered
6	2	3060 Lester St	West Valley City	84119	88kg	10:30 AM	TBD	En Route
7	3	1330 2100 S	Salt Lake City	84106	8kg	EOD	TBD	At Hub
8	2	300 State St	Salt Lake City	84103	9kg	EOD	TBD	En Route
9	3	300 State St	Salt Lake City	84103	2kg	EOD	TBD	At Hub
10	3	600 E 900 South	Salt Lake City	84105	1kg	EOD	TBD	At Hub
11	3	2600 Taylorsville Blvd	Salt Lake City	84118	1kg	EOD	TBD	At Hub
12	3	3575 W Valley Central Station bus Loop	West Valley City	84119	1kg	EOD	TBD	At Hub
13	1	2010 W 500 S	Salt Lake City	84104	2kg	10:30 AM	9:21:41	Delivered
14	1	4300 S 1300 E	Millcreek	84117	88kg	10:30 AM	8:06:21	Delivered
15	1	4580 S 2300 E	Holladay	84117	4kg	9:00 AM	8:13:01	Delivered
16	1	4580 S 2300 E	Holladay	84117	88kg	10:30 AM	8:13:01	Delivered
17	3	3148 S 1100 W	Salt Lake City	84119	2kg	EOD	TBD	At Hub
18	2	1488 4800 S	Salt Lake City	84123	6kg	EOD	TBD	En Route
19	1	177 W Price Ave	Salt Lake City	84115	37kg	EOD	8:31:21	Delivered
20	1	3595 Main St	Salt Lake City	84115	37kg	10:30 AM	8:29:41	Delivered
21	1	3595 Main St	Salt Lake City	84115	3kg	EOD	8:29:41	Delivered
22	3	6351 South 900 East	Murray	84121	2kg	EOD	TBD	At Hub
23	3	5100 South 2700 West	Salt Lake City	84118	5kg	EOD	TBD	At Hub
24	3	5025 State St	Murray	84107	7kg	EOD	TBD	At Hub
25	2	5383 South 900 East #104	Salt Lake City	84117	7kg	10:30 AM	9:13:01	Delivered
26	2	5383 South 900 East #104	Salt Lake City	84117	25kg	EOD	9:13:01	Delivered
27	3	1060 Dalton Ave S	Salt Lake City	84104	5kg	EOD	TBD	At Hub
28	2	2835 Main St	Salt Lake City	84115	7kg	EOD	9:30:21	Delivered
29	1	1330 2100 S	Salt Lake City	84106	2kg	10:30 AM	8:50:01	Delivered
30	1	300 State St	Salt Lake City	84103	1kg	10:30 AM	9:07:41	Delivered
31	1	3365 S 900 W	Salt Lake City	84119	1kg	10:30 AM	9:41:01	Delivered
32	2	3365 S 900 W	Salt Lake City	84119	1kg	EOD	9:40:01	Delivered
33	3	2530 S 500 E	Salt Lake City	84106	1kg	EOD	TBD	At Hub
34	1	4580 S 2300 E	Holladay	84117	2kg	10:30 AM	8:13:01	Delivered
35	3	1060 Dalton Ave S	Salt Lake City	84104	88kg	EOD	TBD	At Hub
36	2	2300 Parkway Blvd	West Valley City	84119	88kg	EOD	TBD	En Route
37	1	410 S State St	Salt Lake City	84111	2kg	10:30 AM	9:04:21	Delivered
38	2	410 S State St	Salt Lake City	84111	9kg	EOD	TBD	En Route
39	3	2010 W 500 S	Salt Lake City	84104	9kg	EOD	TBD	At Hub
40	1	380 W 2880 S	Salt Lake City	84115	45kg	10:30 AM	8:37:01	Delivered

C950 Task-2 WGUPS Write-Up

D3. Third Status Check

All Package Details at [12:30:00]								
ID	Truck	Street	City	Zipcode	Weight	Deadline	Delivered At	Status
1	1	195 W Oakland Ave	Salt Lake City	84115	21kg	10:30 AM	8:40:41	Delivered
2	3	2530 S 500 E	Salt Lake City	84106	44kg	EOD	10:58:21	Delivered
3	2	233 Canyon Rd	Salt Lake City	84103	2kg	EOD	10:42:21	Delivered
4	1	380 W 2880 S	Salt Lake City	84115	4kg	EOD	8:37:01	Delivered
5	1	410 S State St	Salt Lake City	84111	5kg	EOD	9:04:21	Delivered
6	2	3060 Lester St	West Valley City	84119	88kg	10:30 AM	9:45:01	Delivered
7	3	1330 2100 S	Salt Lake City	84106	8kg	EOD	11:03:41	Delivered
8	2	300 State St	Salt Lake City	84103	9kg	EOD	10:44:21	Delivered
9	3	410 S State St	Salt Lake City	84111	2kg	EOD	11:22:21	Delivered
10	3	600 E 900 South	Salt Lake City	84105	1kg	EOD	11:13:01	Delivered
11	3	2600 Taylorsville Blvd	Salt Lake City	84118	1kg	EOD	12:14:21	Delivered
12	3	3575 W Valley Central Station bus Loop	West Valley City	84119	1kg	EOD	TBD	En Route
13	1	2010 W 500 S	Salt Lake City	84104	2kg	10:30 AM	9:21:41	Delivered
14	1	4300 S 1300 E	Millcreek	84117	88kg	10:30 AM	8:06:21	Delivered
15	1	4580 S 2300 E	Holladay	84117	4kg	9:00 AM	8:13:01	Delivered
16	1	4580 S 2300 E	Holladay	84117	88kg	10:30 AM	8:13:01	Delivered
17	3	3148 S 1100 W	Salt Lake City	84119	2kg	EOD	11:57:01	Delivered
18	2	1488 4800 S	Salt Lake City	84123	6kg	EOD	10:03:41	Delivered
19	1	177 W Price Ave	Salt Lake City	84115	37kg	EOD	8:31:21	Delivered
20	1	3595 Main St	Salt Lake City	84115	37kg	10:30 AM	8:29:41	Delivered
21	1	3595 Main St	Salt Lake City	84115	3kg	EOD	8:29:41	Delivered
22	3	6351 South 900 East	Murray	84121	2kg	EOD	10:38:21	Delivered
23	3	5100 South 2700 West	Salt Lake City	84118	5kg	EOD	12:13:01	Delivered
24	3	5025 State St	Murray	84107	7kg	EOD	10:28:01	Delivered
25	2	5383 South 900 East #104	Salt Lake City	84117	7kg	10:30 AM	9:13:01	Delivered
26	2	5383 South 900 East #104	Salt Lake City	84117	25kg	EOD	9:13:01	Delivered
27	3	1060 Dalton Ave S	Salt Lake City	84104	5kg	EOD	11:41:41	Delivered
28	2	2835 Main St	Salt Lake City	84115	7kg	EOD	9:30:21	Delivered
29	1	1330 2100 S	Salt Lake City	84106	2kg	10:30 AM	8:50:01	Delivered
30	1	300 State St	Salt Lake City	84103	1kg	10:30 AM	9:07:41	Delivered
31	1	3365 S 900 W	Salt Lake City	84119	1kg	10:30 AM	9:41:01	Delivered
32	2	3365 S 900 W	Salt Lake City	84119	1kg	EOD	9:40:01	Delivered
33	3	2530 S 500 E	Salt Lake City	84106	1kg	EOD	10:58:21	Delivered
34	1	4580 S 2300 E	Holladay	84117	2kg	10:30 AM	8:13:01	Delivered
35	3	1060 Dalton Ave S	Salt Lake City	84104	88kg	EOD	11:41:41	Delivered
36	2	2300 Parkway Blvd	West Valley City	84119	88kg	EOD	9:50:21	Delivered
37	1	410 S State St	Salt Lake City	84111	2kg	10:30 AM	9:04:21	Delivered
38	2	410 S State St	Salt Lake City	84111	9kg	EOD	10:39:01	Delivered
39	3	2010 W 500 S	Salt Lake City	84104	9kg	EOD	11:36:21	Delivered
40	1	380 W 2880 S	Salt Lake City	84115	45kg	10:30 AM	8:37:01	Delivered

E. Screenshot of Code Execution

```
Cole Linke
ID: 011917160

--- Truck One Details ---
Depart Time: 8:00:01
Return Time: 9:53:21
Total Mileage: 34.0
--- Truck Two Details ---
Depart Time: 9:05:01
Return Time: 11:09:41
Total Mileage: 37.4
--- Truck Three Details ---
Depart Time: 10:20:01
Return Time: 13:06:41
Total Mileage: 50.0
-----
Total Truck Mileage: 121.4

--- EOD Package Details ---
[1] Get all package details
[2] Get a specific package's details
--- Time Specific Package Details ---
[3] Get all package details
[4] Get a specific package's details
-----
[!] Quit Program
[ ]: |
```

F1. Strengths of the Chosen Algorithm

Strength 1: Scalability

The first strength of the algorithm is its scalability. The algorithm is able to handle varying amounts of data without having to make changes to the core logic. For example, the algorithm would be able to accept more or fewer packages, trucks, and addresses.

Strength 2: Self-adjusting

Another strength of the algorithm is that it is self-adjusting. The nearest neighbor logic will automatically recalculate the delivery route based on the truck's location and the number of packages. If a package's target address was changed or the package was removed entirely, the algorithm would adjust to accommodate the changes. It would also adapt to accommodate different truck configurations, such as faster or slower, or those with varying load capacities.

F2. Verification of Algorithm

Verification can be seen in the two screenshots below.

- Total distance traveled < 140
- All packages are delivered by their deadline
- Packages 13, 14, 15, 16, 19, and 20 are all on the same truck
- Packages delayed on flight are on truck 2, which doesn't leave until 9:05 AM
- Packages designated for truck 2 are loaded on truck 2
- Package 9 doesn't depart until 10:20 when its address is corrected

Ex 1:

C950 Task-2 WGUPS Write-Up

```
--- Truck One Details ---  
Depart Time: 8:00:00  
Return Time: 9:53:20  
Total Mileage: 34.0  
--- Truck Two Details ---  
Depart Time: 9:05:00  
Return Time: 11:09:40  
Total Mileage: 37.4  
--- Truck Three Details ---  
Depart Time: 10:20:00  
Return Time: 13:00:00  
Total Mileage: 48.0  
-----  
Total Truck Mileage: 119.4
```

Ex 2:

C950 Task-2 WGUPS Write-Up

All Package Details at [13:00:00]

ID	Truck	Street	City	Zipcode	Weight	Deadline	Delivered At	Status
1	1	195 W Oakland Ave	Salt Lake City	84115	21kg	10:30 AM	8:40:40	Delivered
2	3	2530 S 500 E	Salt Lake City	84106	44kg	EOD	10:58:20	Delivered
3	2	233 Canyon Rd	Salt Lake City	84103	2kg	EOD	10:42:20	Delivered
4	1	380 W 2880 S	Salt Lake City	84115	4kg	EOD	8:37:00	Delivered
5	1	410 S State St	Salt Lake City	84111	5kg	EOD	9:04:20	Delivered
6	2	3060 Lester St	West Valley City	84119	88kg	10:30 AM	9:45:00	Delivered
7	3	1330 2100 S	Salt Lake City	84106	8kg	EOD	11:03:40	Delivered
8	2	300 State St	Salt Lake City	84103	9kg	EOD	10:44:20	Delivered
9	3	410 S State St	Salt Lake City	84111	2kg	EOD	11:19:00	Delivered
10	3	600 E 900 South	Salt Lake City	84105	1kg	EOD	11:13:00	Delivered
11	3	2600 Taylorsville Blvd	Salt Lake City	84118	1kg	EOD	12:07:40	Delivered
12	3	3575 W Valley Central Station bus Loop	West Valley City	84119	1kg	EOD	12:34:40	Delivered
13	1	2010 W 500 S	Salt Lake City	84104	2kg	10:30 AM	9:21:40	Delivered
14	1	4300 S 1300 E	Millcreek	84117	88kg	10:30 AM	8:06:20	Delivered
15	1	4580 S 2300 E	Holladay	84117	4kg	9:00 AM	8:13:00	Delivered
16	1	4580 S 2300 E	Holladay	84117	88kg	10:30 AM	8:13:00	Delivered
17	3	3148 S 1100 W	Salt Lake City	84119	2kg	EOD	11:50:20	Delivered
18	2	1488 4800 S	Salt Lake City	84123	6kg	EOD	10:03:40	Delivered
19	1	177 W Price Ave	Salt Lake City	84115	37kg	EOD	8:31:20	Delivered
20	1	3595 Main St	Salt Lake City	84115	37kg	10:30 AM	8:29:40	Delivered
21	1	3595 Main St	Salt Lake City	84115	3kg	EOD	8:29:40	Delivered
22	3	6351 South 900 East	Murray	84121	2kg	EOD	10:38:20	Delivered
23	3	5100 South 2700 West	Salt Lake City	84118	5kg	EOD	12:06:20	Delivered
24	3	5025 State St	Murray	84107	7kg	EOD	10:28:00	Delivered
25	2	5383 South 900 East #104	Salt Lake City	84117	7kg	10:30 AM	9:13:00	Delivered
26	2	5383 South 900 East #104	Salt Lake City	84117	25kg	EOD	9:13:00	Delivered
27	3	1060 Dalton Ave S	Salt Lake City	84104	5kg	EOD	11:35:00	Delivered
28	2	2835 Main St	Salt Lake City	84115	7kg	EOD	9:30:20	Delivered
29	1	1330 2100 S	Salt Lake City	84106	2kg	10:30 AM	8:50:00	Delivered
30	1	300 State St	Salt Lake City	84103	1kg	10:30 AM	9:07:40	Delivered
31	1	3365 S 900 W	Salt Lake City	84119	1kg	10:30 AM	9:41:00	Delivered
32	2	3365 S 900 W	Salt Lake City	84119	1kg	EOD	9:40:00	Delivered
33	3	2530 S 500 E	Salt Lake City	84106	1kg	EOD	10:58:20	Delivered
34	1	4580 S 2300 E	Holladay	84117	2kg	10:30 AM	8:13:00	Delivered
35	3	1060 Dalton Ave S	Salt Lake City	84104	88kg	EOD	11:35:00	Delivered
36	2	2300 Parkway Blvd	West Valley City	84119	88kg	EOD	9:50:20	Delivered
37	1	410 S State St	Salt Lake City	84111	2kg	10:30 AM	9:04:20	Delivered
38	2	410 S State St	Salt Lake City	84111	9kg	EOD	10:39:00	Delivered
39	3	2010 W 500 S	Salt Lake City	84104	9kg	EOD	11:29:40	Delivered
40	1	380 W 2880 S	Salt Lake City	84115	45kg	10:30 AM	8:37:00	Delivered

F3. Other Possible Algorithms

1. Dijkstra
2. Bellman-Ford

F3a. Algorithm Differences

Dijkstra and Bellman-Ford both find the shortest distance between two points on a graph. The algorithm I used finds the nearest point at each step and continues until it's out of steps or all packages are delivered. Dijkstra and Bellman-Ford would work better if the trucks had fixed routes or delivery areas, or if weights like traffic were included in the scenario.

G. Different Approach

If I were to do this project over again, I would try to make the package loading more efficient. Currently, the packages are loaded manually, but if I were to implement an automated way of loading the packages, it would improve the scalability of the program.

I would have also liked to add another layer of customization to the UI. In the current version of the program, the hash table is only sorted by ID. Letting the user sort the data in different ways would make the program more user-friendly and could help with data analysis.

H. Verification of Data Structure

Data is able to be inserted into the hash table using the `insert()` function, and it can also use the `search()` function to look up said data. The data includes all the required components (address, deadline, city, etc.).

H1. Other Data Structures

1. Weighted Graph
2. Priority Queue

H1a. Data Structure Differences

A weighted graph could be used to store all addresses as nodes and the distances between them as weighted edges. The hash table in the solution only stores package information, not map or distance data.

A priority queue could be used to manage packages based on their delivery deadlines or importance. This would ensure that time-sensitive packages are delivered first. The hash table in the solution stores deadlines but does not automatically prioritize packages based on them.

I. Sources

Sources used to help me develop my code.

ANSI Escape sequences. (n.d.). ASCII-Table.

<https://web.archive.org/web/20201214113226/http://asciitable.com/ansi-escape-sequences.php>

Built-in functions. (n.d.). Python Documentation.

<https://docs.python.org/3/library/functions.html#sorted>

CSV — CSV file reading and writing. (n.d.). Python Documentation.

<https://docs.python.org/3/library/csv.html#csv.reader>

datetime — Basic date and time types. (n.d.). Python Documentation.

<https://docs.python.org/3/library/datetime.html#strptime-and-strptime-behavior>

Sorting techniques. (n.d.). Python Documentation.

<https://docs.python.org/3/howto/sorting.html>

J. Professional Communication
